HADIYA ZONE SCIENCE & INFORMATION TECHNOLOGY DEPARTMENT

Master API Specification & Architecture Blueprint

Complete Contract: Letter Management System (LCMS) & Public Portal CMS

VERSION 2.0 • PRODUCTION CONTRACT

Stack: Express + Prisma + TypeScript + Socket.IO

Target: Backend Engineering Team

1. System Scope & Core Domains

This technical contract defines the end-to-end API specification for the HZSITD platform across its two core operational pillars:

- **Letter Management System (LCMS):** Multi-tier hierarchical correspondence, cryptographic verification QR codes, minute directives, departmental delegations, attachment handling, and real-time Socket.IO synchronization.
- **Portal Management System (CMS & Public Gateway):** Public announcements, zonal digital initiatives, citizen contact inquiry tickets, department directories, and cryptographic verification ledger.

2. Role Hierarchy & Authority Matrix

Role	Authority Level	Capabilities in Workflow
Department Head	Executive (L3)	Final endorsement & legal seal; minutes & downward directives to directorates or specific officers; zone-wide audit clearance.
Directory Lead	Managerial (L2)	Clears subordinate officer drafts; minutes & delegates directives to officers within their directorate; inter-directorate correspondence.
Officer	Operational (L1)	Authoring workbench; executes assigned tasks/directives; uploads evidence/PDF attachments; submits drafts up to lead.
System Admin	Governance (L0)	User provisioning, directory setup, system settings, immutable audit log monitoring.

3. Standard Envelope & Error Conventions

```
// Success Envelope:
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": { ... },
  "pagination": { "page": 1, "limit": 20, "total": 85, "totalPages": 5 }
}

// Error Envelope:
{
  "success": false,
  "error": {
    "code": "PROTOCOL_VIOLATION",
    "message": "An author cannot address an internal directive to themselves.",
    "fieldErrors": [{ "field": "assigned_officer_id", "message": "Invalid recipient" }]
  }
}
```

4. Complete Prisma Schema Model Structure

Below is the complete relational model mapping for your `schema.prisma`:

```
enum UserRole {
  OFFICER          @map("Officer")
  DIRECTORY_LEAD   @map("Directory Lead")
  DEPARTMENT_HEAD  @map("Department Head")
  SYSTEM_ADMIN     @map("System Admin")
}

enum LetterDirection {
  INCOMING
  OUTGOING
}

enum LetterPriority {
  ROUTINE       @map("Routine")
  NORMAL        @map("Normal")
  URGENT        @map("Urgent")
  CONFIDENTIAL  @map("Confidential")
}

enum LetterStatus {
  DRAFT                     @map("Draft")
  UNDER_REVIEW              @map("Under Review")
  PENDING_DEPARTMENT_HEAD   @map("Pending Department Head")
  APPROVED                  @map("Approved")
  DISPATCHED                @map("Dispatched")
  RETURNED_FOR_REVISION     @map("Returned for Revision")
  ARCHIVED                  @map("Archived")
}

enum ApprovalStage {
  OFFICER_DRAFT               @map("Officer Draft")
  DIRECTORY_LEAD_REVIEW       @map("Directory Lead Review")
  DEPARTMENT_HEAD_ENDORSEMENT @map("Department Head Endorsement")
  COMPLETED                   @map("Completed")
  RETURNED                    @map("Returned")
}

model Directory {
  id          Int      @id @default(autoincrement())
  code        String   @unique
  name        String
  shortName   String?  @map("short_name")
  users       User[]
  sentLetters Letter[] @relation("SenderDirectory")
  recvLetters Letter[] @relation("RecipientDirectory")
}

model User {
  id            Int       @id @default(autoincrement())
  username      String    @unique
  passwordHash  String    @map("password_hash")
  fullName      String    @map("full_name")
  email         String    @unique
  role          UserRole
  positionTitle String    @map("position_title")
  directoryId   Int?      @map("directory_id")
  directory     Directory? @relation(fields: [directoryId], references: [id])
}

model Letter {
  id                    Int             @id @default(autoincrement())
  referenceNo           String          @unique @map("reference_no")
  trackingToken         String?         @unique @map("tracking_token")
  direction             LetterDirection
```

```
  subject               String
  body                  String
  priority              LetterPriority  @default(ROUTINE)
  letterType            String          @default("Official Letter") @map("letter_type")
  senderType            String          @default("INTERNAL") @map("sender_type")
  senderId              Int?            @map("sender_id")
  senderDirectoryId     Int?            @map("sender_directory_id")
  recipientType         String          @default("INTERNAL") @map("recipient_type")
  recipientDirectoryId  Int?            @map("recipient_directory_id")
  assignedOfficerId     Int?            @map("assigned_officer_id")
  status                LetterStatus    @default(DRAFT)
  approvalStage         ApprovalStage   @default(OFFICER_DRAFT) @map("approval_stage")
  digitalSignature      String?         @map("digital_signature")
  stampData             String?         @map("stamp_data")
  timeline              LetterTimeline[]
  comments              LetterComment[]
  attachments           Attachment[]
}
```

5. RESTful Endpoint Catalog

Module 1: Authentication & Governance

Method	Endpoint	Description & Access Role
POST	`/api/auth/login`	Authenticates username/password; returns JWT + user profile.
GET	`/api/auth/me`	Validates bearer token; returns current authenticated session.
GET	`/api/users`	Returns personnel roster filtered by `directory_id` or `role`.
GET	`/api/directories`	Returns list of all 9 institutional directorates.

Module 2: Letter Management & State Engine

Method	Endpoint	Description & Access Role
GET	`/api/letters`	Paginated correspondence filtered by tab (`inbox`, `sent`, `drafts`, `archived`).
GET	`/api/letters/:id`	Complete relational letter with attachments, minute trail, and signatures.
POST	`/api/letters`	Creates letter draft or direct submission with target directorate/officer.
POST	`/api/letters/:id/action`	Executes state transition: `submit_to_lead`, `approve_to_head`, `endorse_and_seal`, `return_for_revision`, `assign_to_directory`, `delegate_to_officer`.
POST	`/api/letters/:id/comments`	Appends minute instruction or review note to letter trail.
POST	`/api/upload`	Uploads multipart PDF or evidence attachment.

Module 3: Verification & Public Cryptographic Ledger

Method	Endpoint	Description & Access Role
GET	`/api/public/letters/verify/:token`	QR Code scan endpoint: returns authenticity certificate, seal timestamp, and validity.
GET	`/api/public/letters/track?ref=...`	Public reference lookup service.

Module 4: Portal CMS & Public Operations

Method	Endpoint	Description & Access Role
GET	`/api/portal/announcements`	Public news, circulars, and official press releases.
POST	`/api/portal/announcements`	Create announcement (Admin / Department Head only).
GET	`/api/portal/projects`	Public zonal technology initiatives showcase.
POST	`/api/portal/inquiries`	Public contact form submission; issues tracking ticket number.

> **Working Reference Backend Available:** A fully functional Express reference server with SQLite/Prisma compatibility is located at `HZSITD/backend/server.js` with sample database queries in `HZSITD/backend/db.js`.